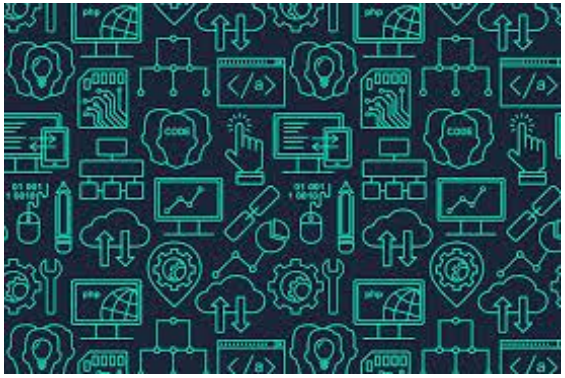
INF 1001
INTRODUCTION TO COMPUTING

Combinatorial Circuits

Lecture 3

Prof. Vinod Prasad
Email: vinod.prasad@singaporetech.edu.sg
https://apvinod.github.io



Outline

- Basic logic gates and Boolean algebra
- Combinatorial circuits
  - Truth-table
  - Logic equations
  - Logic simplification

Logic Gates

- The building blocks used to create digital circuits are **logic gates.**
- There are three elementary logic gates and a range of other simple gates. Each gate has its own **logic symbol** which allows complex functions to be represented by a logic diagram.
- The function of each gate can be represented by a **truth table** or using **Boolean notation** (George Boole, British Mathematician, 1815-1864).

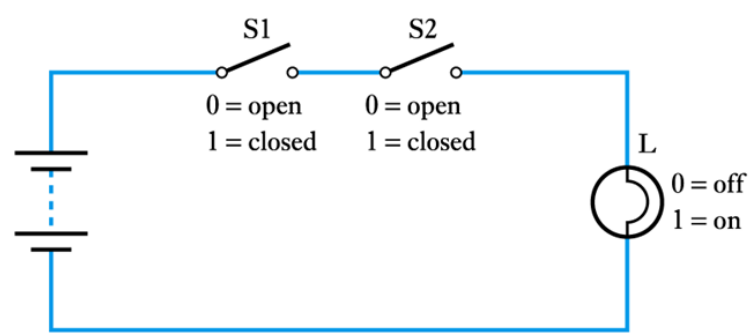
The AND gate

Boolean expression: C = A.B

(a) Circuit symbol

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

(b) Truth table



Equivalent circuit: Lamp is ON only if both S1 **and** S2 are closed (ON).

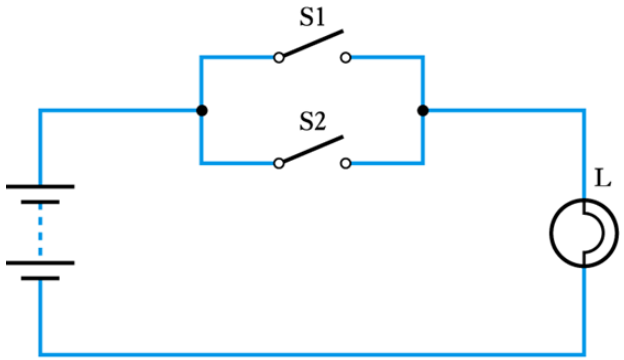
The OR gate

Boolean expression: C = A+B

(a) Circuit symbol

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

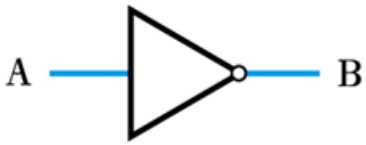
(b) Truth table



Equivalent circuit: Lamp is ON if S1 **or** S2 is closed (ON).

The NOT gate (or inverter)

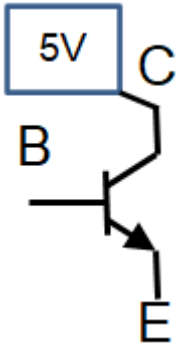
Boolean expression: $B = \overline{A}$



(a) Circuit symbol

A	B
0	1
1	0

(b) Truth table



Equivalent circuit: Transistor (output is inverted).

The NAND gate

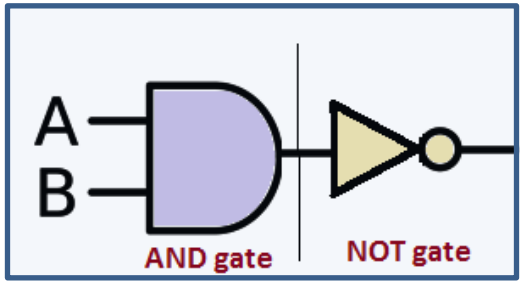
Boolean expression: $C = \overline{A \cdot B}$

(a) Circuit symbol

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

(b) Truth table

NAND is inverted AND



NAND gate

The NOR gate

Boolean expression: $C = \overline{A + B}$

(a) Circuit symbol

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

(b) Truth table

NOR is inverted OR

NOR gate

The Exclusive OR (XOR) gate

Boolean expression: $C = A \oplus B$ $\quad A \oplus B = A\bar{B} + \bar{A}B$

(a) Circuit symbol

A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

(b) Truth table

Output is TRUE if either of its inputs are TRUE (1), but not if both inputs are TRUE (1) / FALSE (0).

The Exclusive NOR (XNOR) gate

Boolean expression: $C = \overline{A \oplus B}$

(a) Circuit symbol

A	B	C
0	0	1
0	1	0
1	0	0
1	1	1

(b) Truth table

$\overline{A \oplus B} = AB + \bar{A}\,\bar{B}$

Boolean Algebra

- **Boolean Variables**
  - variables that can only take the vales ‘0’ or ‘1’
- **Boolean Functions**
  - each of the logic functions (such as AND, OR and NOT) are represented by symbols as described above
- **Boolean Theorems** - a set of **identities** and **laws**

Boolean algebra will enable you to optimize circuits.

Boolean identities

AND Function	OR Function	NOT function
$0 \bullet 0 = 0$	$0+0=0$	$\overline{0} = 1$
$0 \bullet 1 = 0$	$0+1=1$	$\overline{1} = 0$
$1 \bullet 0 = 0$	$1+0=1$	$\overline{\overline{A}} = A$
$1 \bullet 1 = 1$	$1+1=1$	
$A \bullet 0 = 0$	$A+0=A$	
$0 \bullet A = 0$	$0+A=A$	
$A \bullet 1 = A$	$A+1=1$	
$1 \bullet A = A$	$1+A=1$	
$A \bullet A = A$	$A+A=A$	
$A \bullet \overline{A} = 0$	$A + \overline{A} = 1$	

Boolean laws

Commutative law $AB = BA$ $A + B = B + A$	**Absorption law** $A + AB = A$ $A(A+B) = A$
Distributive law $A(B+C) = AB + BC$ $A + BC = (A+B)(A+C)$	**De Morgan's law** $\overline{A+B} = \overline{A} \bullet \overline{B}$ $\overline{A \bullet B} = \overline{A} + \overline{B}$
Associative law $A(BC) = (AB)C$ $A + (B+C) = (A+B) + C$	

A(1+B) = A(1) = A

A(A+B) = A.A+A.B
= A + A.B
= A(1+B)
= A (as 1+B = 1)

Verification of De Morgan's law

A	B	$\overline{A}$	$\overline{B}$	A·B	$\overline{A \cdot B}$	$\overline{A} + \overline{B}$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

A	B	$\overline{A}$	$\overline{B}$	A+B	$\overline{A+B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

$$
\begin{aligned}
A + B.C &= A.1 + B.C \; [Since,\; A.1 = A] \\
&= A.(1+B) + B.C \; [Since,\; B+1 = 1] \\
&= A.1 + AB + BC \\
&= A.(1+C) + AB + BC \; [Since,\; A.A = A.1 = A] \\
&= A.(A+C) + B.(A+C) \\
&= (A+B).(A+C)
\end{aligned}
$$

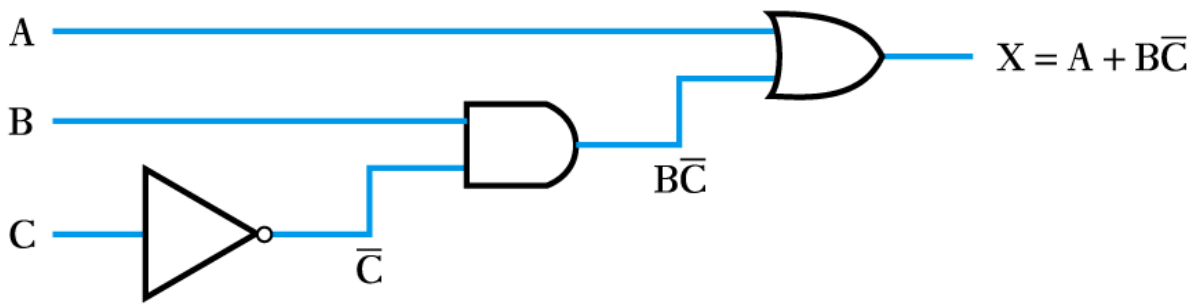
Proof is only for reference (not for exam)

Combinational and Sequential Logic Circuits

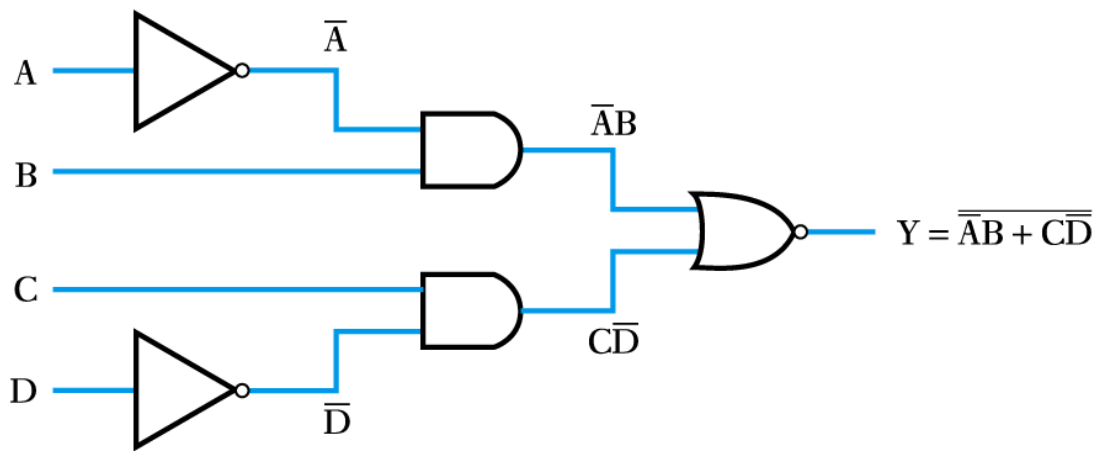
- Digital systems may be divided into two broad categories:
  - **Combinational logic circuit**
    - Outputs are determined solely by the current states of the inputs.
    - Uses logic gates only.
  - **Sequential logic circuit**
    - Outputs are determined not only by the current inputs but also by the past sequence of inputs (stored in storage elements).
    - Uses flip flops.

Boolean Expression Implementation using logic gates

Example: Implement $X = A + B\overline{C}$

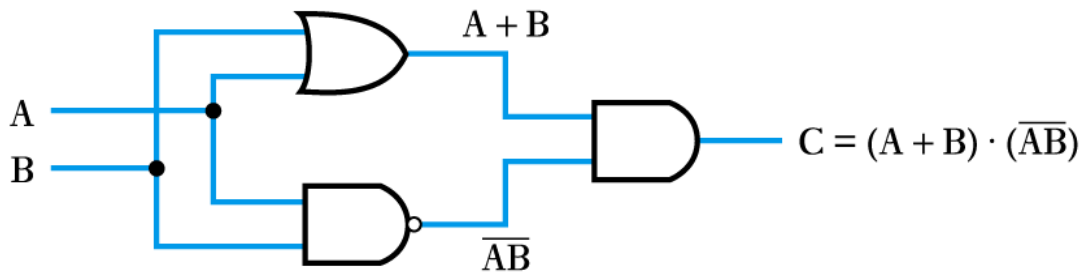


Example: Implement $Y = \overline{\overline{A}B + C\overline{D}}$

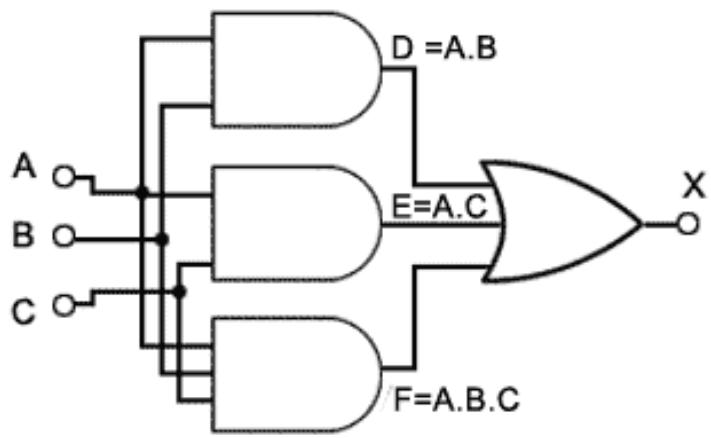


Boolean Expression Implementation using logic gates

Example: Implement $X = (A+B) \bullet \overline{(AB)}$



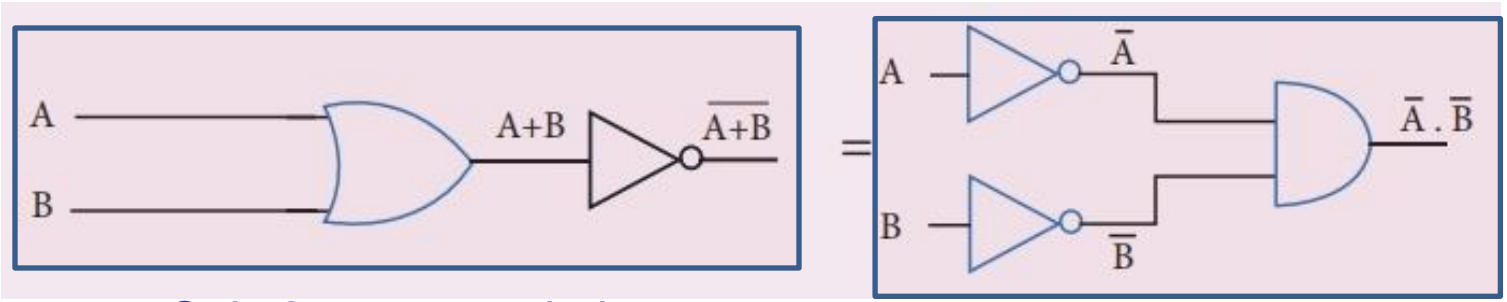
Example: Implement X= (A.B)+(A.C)+(A.B.C)



X= (A.B)+(A.C)+(A.B.C)

Boolean Expression Implementation using logic gates

De Morgan's first law: $\overline{A+B}=\overline{A}\bullet\overline{B}$

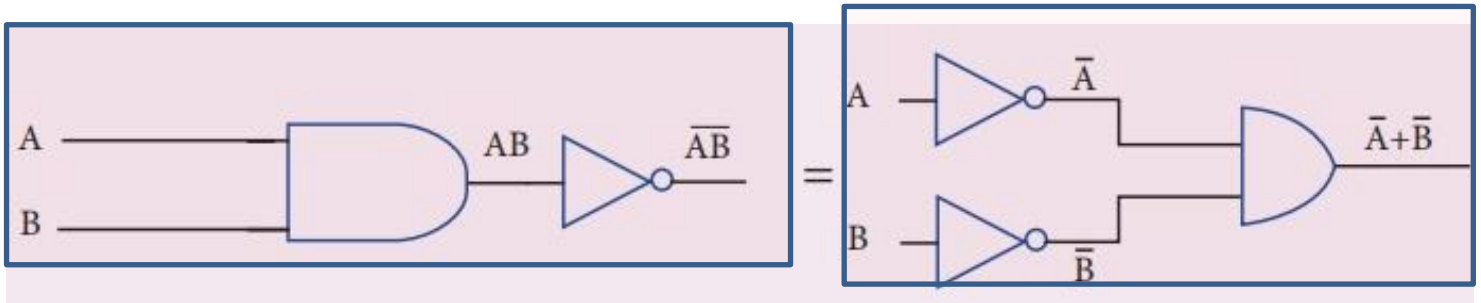


Only 2 gates needed

3 gates needed

So, change $\overline{A}\bullet\overline{B}$ to $\overline{A+B}$ during Boolean simplification to save gates.

De Morgan's second law: $\overline{A\bullet B}=\overline{A}+\overline{B}$



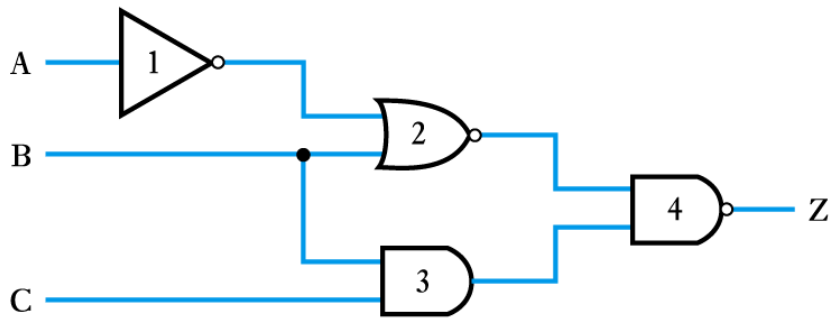
Only 2 gates needed

3 gates needed

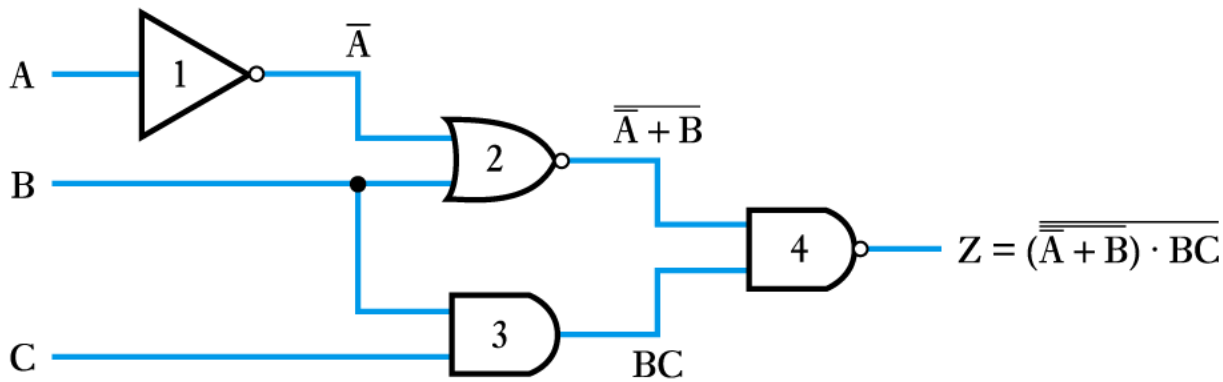
So, change $\overline{A}+\overline{B}$ to $\overline{A\bullet B}$ during Boolean simplification to save gates.

Generating a Boolean expression from a logic circuit

Example:



Work progressively from the inputs to the output adding logic expressions to the output of each gate in turn.



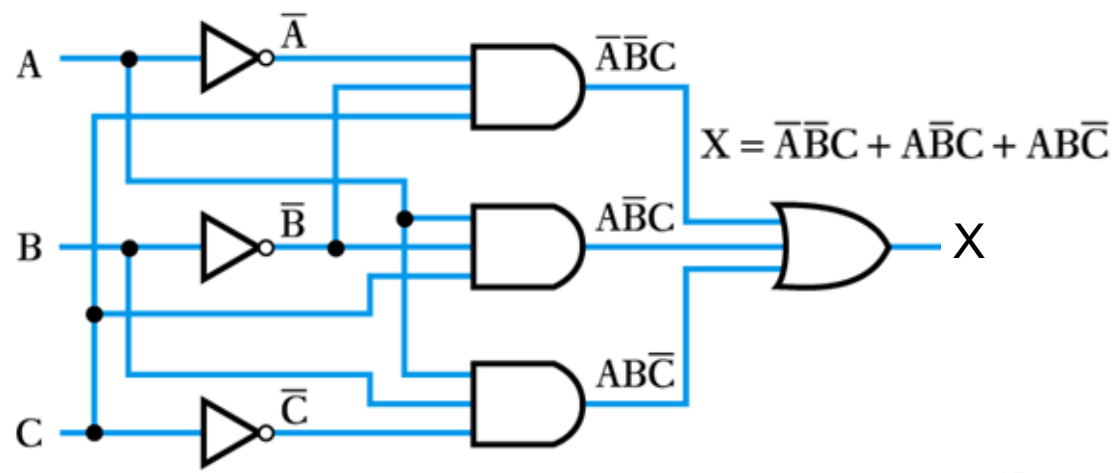
Implementing a logic function from a truth table

Example Implement the function of the following truth table:

A	B	C	X	
0	0	0	0	
0	0	1	1	$\bar{A}\bar{B}C$
0	1	0	0	
0	1	1	0	
1	0	0	0	
1	0	1	1	$A\bar{B}C$
1	1	0	1	$AB\bar{C}$
1	1	1	0	

- Write down a Boolean expression for the output '1'.
- Add the Boolean expressions
- Implement as before

$$X = \bar{A}\bar{B}C + A\bar{B}C + AB\bar{C}$$



Circuit simplification using Boolean algebra

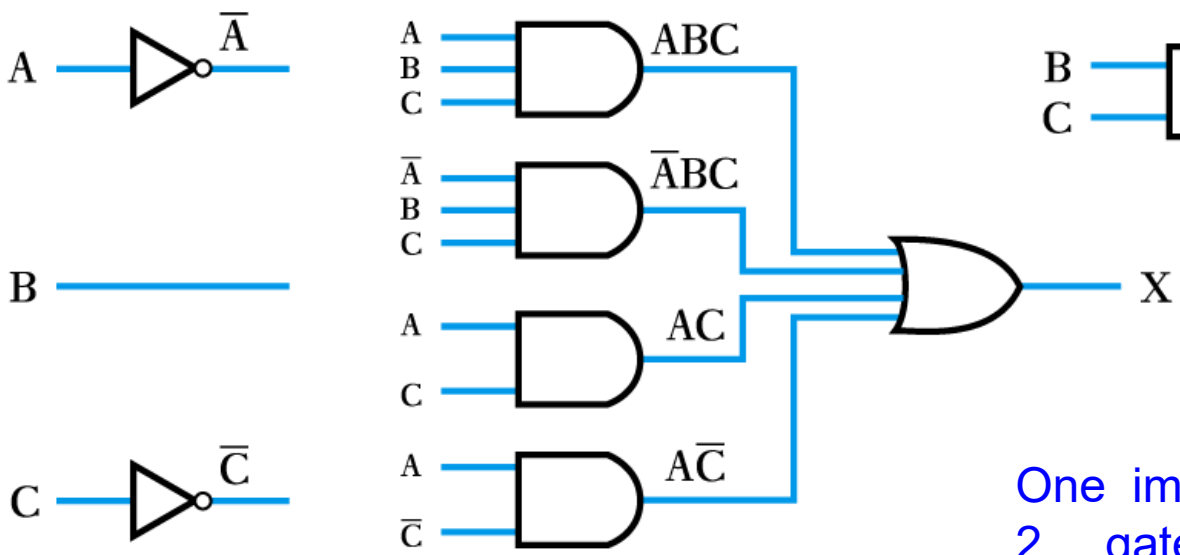
It is possible to *simplify* logic expressions using Boolean algebra.

Example $X = ABC + \bar{A}BC + AC + A\bar{C}$

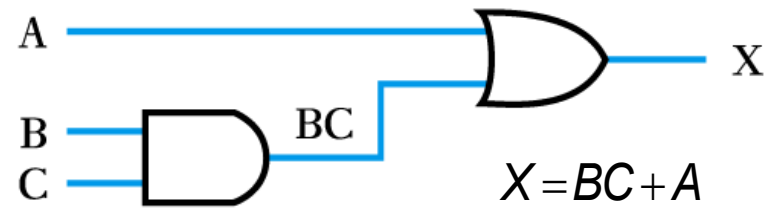
It can be simplified to: $X = BC + A$

$$ABC + \bar{A}BC + AC + A\bar{C} = BC(A + \bar{A}) + A(C + \bar{C}) \quad = BC(1) + A(1) = BC + A$$

$$(A + \bar{A} = 1) \quad (C + \bar{C} = 1)$$



$X = ABC + \bar{A}BC + AC + A\bar{C}$



$X = BC + A$

Two circuits are equivalent. Which is better? Why?

One implementation requires only 2 gates whereas the other implementation requires 5 gates (actually 7 gates, including two NOT gates)!

Truth table, Boolean expressions and Implementations

Example $F_2 = \bar{x}\bar{y}z + \bar{x}yz + x\bar{y}$

$$= \bar{x}z(\bar{y} + y) + x\bar{y}$$

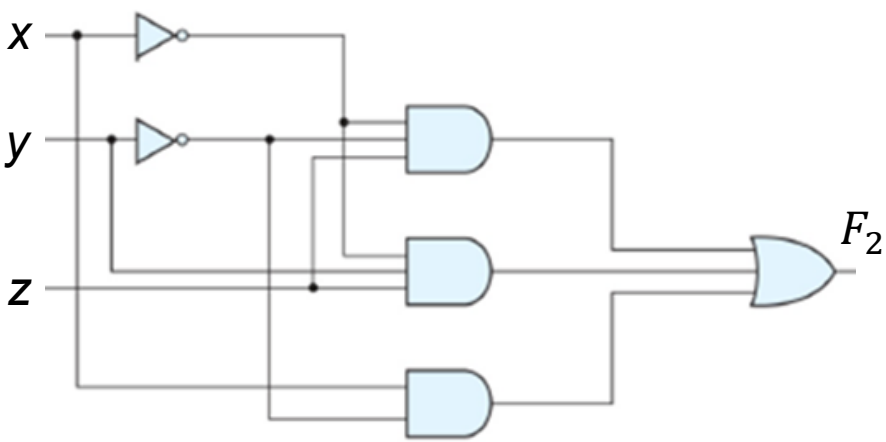
$$= \bar{x}z + x\bar{y} \quad (\text{as } \bar{y} + y = 1)$$

There is only one way that a Boolean function can be represented in a truth table. However, in Boolean algebra, it can be expressed in a variety of ways.

Truth table for F_2

x	y	z	F_2
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

Implementation of $F_2 = \bar{x}\bar{y}z + \bar{x}yz + x\bar{y}$



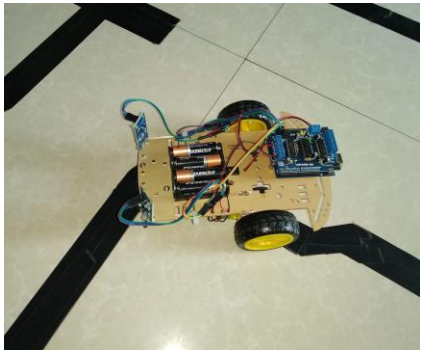
Implementation of $F_2 = \bar{x}z + x\bar{y}$

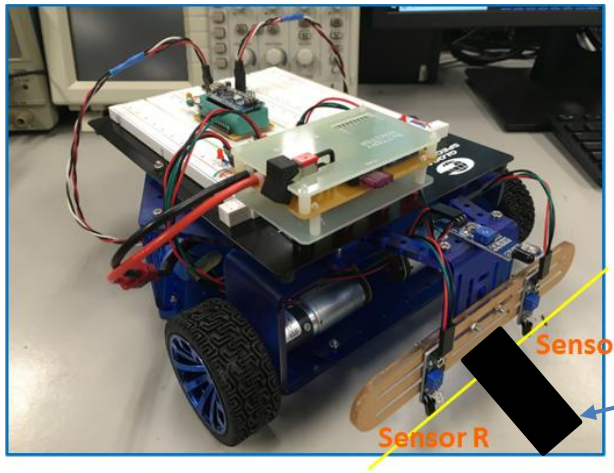
Two circuits are equivalent. Which is better? Why?

Boolean simplification can reduce the number of gates required and the number of inputs to the gates. This is the basic trick behind compact system design.

Digital Logic Circuit Design – A Real World Example

Line-Follower Robot

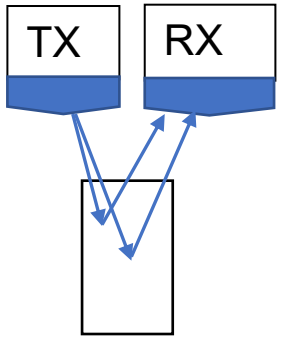




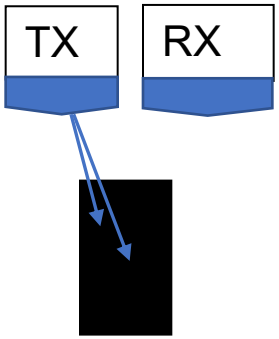
Line-Following Infrared (IR) Sensors (Left and Right)

Center track (black line)

Each IR sensor has a Transmitter (TX) and a Receiver (RX).



White surface reflects IR light (RX receives reflected light)



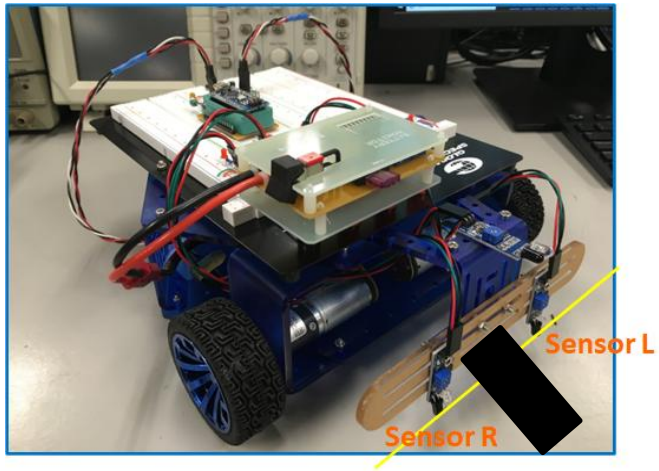
Black surface absorbs IR light (RX does not receive light)

Case 1: When the robot is moving in correct path along the center, the IR sensors are aligned at the top of white surface. RX of 'L' and 'R' sensors receive reflected light and the transistors T_L and T_R connected to them turns ON → Outputs of both transistors become LOW ($T_L = 0$, $T_R = 0$).

This indicates that the robot is in in correct track and the controller will turn ON LEFT and RIGHT MOTORS of the robot M_L=1 and M_R =1 → Robot moves forward.

Digital Logic Circuit Design – A Real World Example

Line-Follower Robot



Case-2: When the robot shifts to the left, the 'L' sensor is still on white surface but the 'R' sensor is on top of black surface. RX of 'L' receives light whereas RX of 'R' does not receive light. T_L connected to 'L' sensor is LOW (0) and T_R connected to 'R' sensor is HIGH (1) → (T_L = 0, T_R = 1).

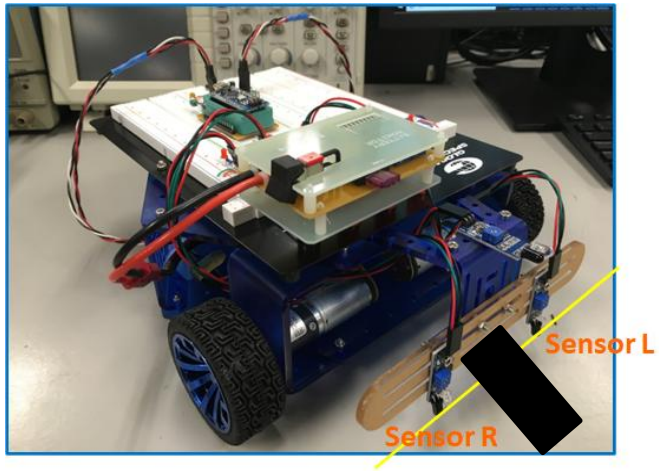
This indicates that the robot is shifting towards left of the track → controller will turn OFF RIGHT MOTOR and turn ON LEFT MOTOR M_L=1 and M_R =0 → Robot moves towards RIGHT back to the center-line.

Case-3: When the robot shifts to the right, the 'R' sensor is still on white surface but the 'L' sensor is on top of black surface. RX of 'R' receives light whereas RX of 'L' does not receive light. T_L connected to 'L' sensor is HIGH (1) and T_R connected to 'R' sensor is LOW (0) → (T_L = 1, T_R = 0).

This indicates that the robot is shifting towards right of the track → controller will turn OFF LEFT MOTOR and turn ON RIGHT MOTOR M_L=0 and M_R =1 → Robot moves towards LEFT back to the center-line.

Digital Logic Circuit Design – A Real World Example

Line-Follower Robot



Case-4: If the robot makes a 90-deg turn to LEFT or RIGHT, both 'L' and 'R' sensors will be aligned on top of black surface. RX of 'L' and 'R' do not receive light. T_L connected to 'L' sensor is HIGH (1) and T_R connected to 'R' sensor is HIGH (1) → (T_L = 1, T_R = 1).

This indicates that the robot has made a 90-deg turn away from the track → controller will turn OFF both RIGHT MOTOR and LEFT MOTOR M_L=0 and M_R =0 → Robot will STOP.

TRUTH TABLE

RX-Sensor L	RX-Sensor R	T_L – Sensor L	T_R – Sensor R	Status	LEFT Motor	RIGHT Motor	Control action
light	light	0	0	Straight track	1	1	Move forward
light	No light	0	1	Shift LEFT	1	0	Turn RIGHT
No light	light	1	0	Shift RIGHT	0	1	Turn LEFT
No light	No light	1	1	90-deg turn	0	0	STOP

Digital Logic Circuit Design – A Real World Example

Line-Follower Robot

TRUTH TABLE

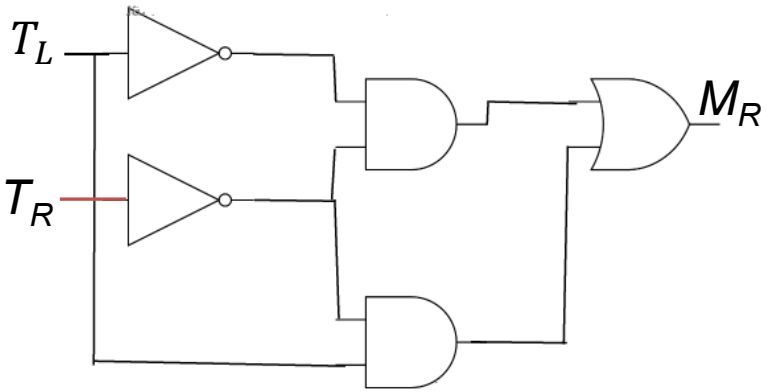
T_L	T_R	LEFT Motor (M_L)	RIGHT Motor (M_R)
0	0	1	1
0	1	1	0
1	0	0	1
1	1	0	0

BOOLEAN EXPRESSIONS

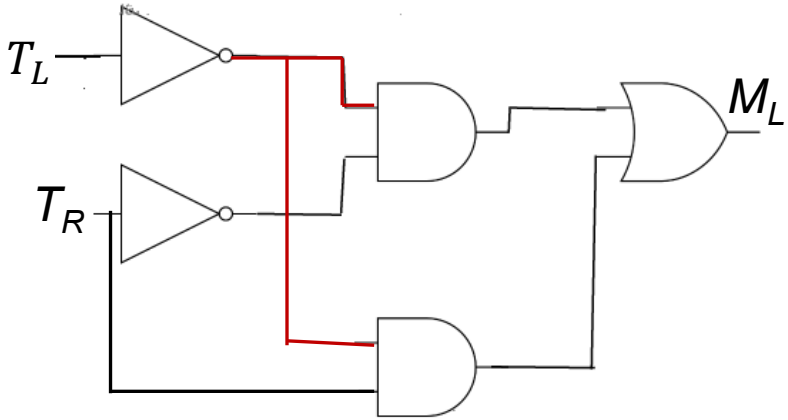
$$M_L = = \overline{T_L}.\overline{T_R} + \overline{T_L}.T_R$$

$$M_R = = \overline{T_L}.\overline{T_R} + T_L.\overline{T_R}$$

Logic circuit: $M_R = = \overline{T_L}.\overline{T_R} + T_L.\overline{T_R}$



Logic circuit: $M_L = = \overline{T_L}.\overline{T_R} + \overline{T_L}.T_R$



Can you simplify this circuit using Boolean algebra?

Summary

- Basic logic circuits (AND, OR, NOT)
- Boolean algebra (associative, distributive, De Morgan)
- Logic Circuits, simplification
- Real World application